QUEENSLAND UNIVERSITY OF TECHNOLOGY
SCHOOL OF ELECTRICAL ENGINEERING AND ROBOTICS

DEVELOPMENT OF MPPT CONTROL ALGORITHM

Author: Nathan Hui
Supervisor: Associate Professor Geoff Walker

A Project Paper
Presented to the
Queensland University of Technology
School of Electrical Engineering and Robotics

In Partial Fulfilment
of the Requirements for the Degree of
Bachelor of Engineering (Mechatronics)

CONTENTS

1 Abstract **4**

2 Background and Objectives **5**
2.1 Introduction 5
2.2 Literature Review 6
2.2.1 Maximum Power Point Trackers (MPPT) 10
2.2.2 MPP Tracking Techniques 11
2.3 Research Objectives 12
2.3.1 Proposed Solution 12
2.3.2 Updated Proposal 12

3 Design **13**
3.1 Methodology 13
3.2 Simulation Tools 14
3.3 Stage 1: Linearised Model 14
3.4 Stage 2: Non-linearised PV Module 18
3.5 Stage 3: Shuffle Converters 21
3.6 Summary 24

4 Project Timeline **25**

5 Conclusion and Recommendations **26**

References **27**

A Appendices **30**
A.1 Cell Groupings of Solar Array 30
A.2 Sub-division of Solar Array 31
A.3 Cell Count of Modules 32
A.4 Schematic Diagram of Bypass Diode Rig 33
A.5 Photograph of Bypass Diodes 34

A Final MATLAB Interface Script for Simulink **35**

LIST OF FIGURES

1 Photo of ATN's Priscilla . . . 5
2 Single-Diode Model of PV Cell incl. Ideal Model [1] . . . 6
3 I-V Curve of Single-Diode Model with Points of Interest [2] . . . 7
4 Comparison of SDM and Linearised Models [3] . . . 7
5 Experimental Data of A Cell at Varying Irradiations [1] . . . 8
6 Diagram of Partially Shaded Solar Module with Bypass Diode [4] . . . 9
7 Example IV Curve of Partially Shaded Solar Module [5] . . . 9
8 Various DMPPT Architectures [6] . . . 10
9 Various Partial Power Processing Architectures [7] . . . 11
10 Flowchart of Design Methodology . . . 13
11 Simple Simulink Model of MPPT . . . 14
12 Classic Stateflow Chart in Simulink . . . 15
13 PV Power and Duty Cycle for Simple P&O Model . . . 16
14 Voltage and Current for Simple P&O Model . . . 17
15 I-V Curve Generated By Simulink . . . 18
16 Simulink Model with PV Array Blocks . . . 19
17 Static Irradiance Tracking . . . 19
18 Plot of Test Irradiance . . . 20
19 Results of NL Model With Test Irradiance . . . 20
20 Simulink Model of PV System With Shuffle Converters . . . 21
21 Sampled Irradiance Data . . . 22
22 String Voltages of 3rd Module: Raw . . . 22
23 String Voltages of 3rd Module: Equalised . . . 22
24 Power Output Comparison of 3rd Module . . . 23
25 Power over Time For Full Model . . . 23
26 Closeup of Full Model Power . . . 24
27 Closeup of Individual Voltages in FM . . . 24
28 Gantt Chart of Project . . . 25
29 Module Positions on Array . . . 30
30 Subdivision of Arrays into Strings . . . 31

1 ABSTRACT

This report presents the work completed to model a differential-power-processing (DPP) maximum power point tracker (MPPT). The device, also known as a shuffling converter, was to be installed in a high-performing electric vehicle equipped with a photo-voltaic system, for participation in the Bridgestone World Solar Challenge 2019. Following the ultimate failure of the shufflers during the challenge, further research plans were made to ensure the safety of the devices. The contents of this report explores the research and design of a Simulink model which implements both the central and distributed MPPT modules in the automotive system.

In conjunction with MATLAB, the model was successfully able to demonstrate both the tracking techniques of the shuffle converters as well as the more traditional centralised P&O MPPT. It was through these tools that the effectiveness of the voltage-equalising shuffle converters was shown. These models and MATLAB interfaces grant users to rapidly design and tune PV systems for verification prior to, or in parallel with, hardware and firmware development.

However, this paper alone does not present ample insight for a direct line into hardware implementations. This paper only outlines the design of the simulation tools which may form the groundwork of any future testing, and presents the findings. Future testing and research will require tightly coupled feedback loops between data collection and simulation to accurately and effectively characterise the practical limits of shuffle converters' operations.

2 Background and Objectives

2.1 Introduction

In pursuit of clean and renewable energy, many industries are resorting to photo-voltaic (PV) systems to harness the power of the Sun. The automotive industry is slowly joining the solar train; since 1987 Bridgestone's World Solar Challenge (BWSC), the world's most renowned solar-powered car race, has challenged teams from all around the world to design a vehicle capable of traveling 3,000km powered only by the Sun, in the hopes of achieving sustainable road transport [8]. Along with 50 other teams, Queensland University of Technology (QUT) and other associated universities of the Australian Technology Network (ATN) will participate in the 2019 BWSC.

Given that the vehicle is to primarily source its energy from the Sun, it is critical that the solar array installed on the vehicle is optimised to provide as much power as possible as each vehicle's PV cell area is limited according to the Race Regulations [8].

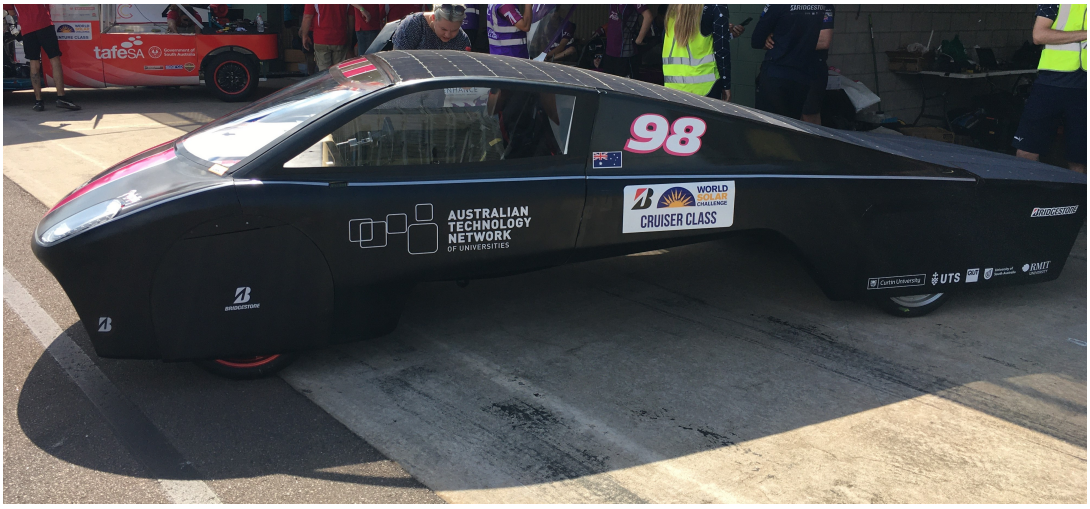


Figure 1: Photo of ATN's Priscilla

The vehicle, ATN Priscilla (pictured above), has an integrated solar array comprised of SunPower Maxeon$_{TM}$ Gen III cells, producing a theoretical maximum power output of 1.253kW. However, the average output in reality will be less than this figure. Not only due to the curvature of the vehicle (and thus, difference in irradiance across the array), but also because of obstacles that may partially shade the array whilst driving.

This project will focus on the implementation of a maximum power point tracker (MPPT) that operates using differential-power-processing (DPP) techniques, otherwise known as a shuffling converter [9].

2.2 Literature Review

PV cell model

The photovoltaic cell, or solar cell, is a device that can convert solar energy into electrical energy. An ideal system comprises of a current source in parallel with a diode, where the current is directly proportional to the intensity of an incident light. However, this model does not accurately reflect the device in practice [1]. Figure 2 shows a single-diode model (SDM) of a PV cell, which is formed by the ideal model as well as any shunt and series resistances ($\mathbf{R_p}$ or $\mathbf{R_{sh}}$ and $\mathbf{R_s}$). Shunt resistances represent high-conductivity paths in parallel with the cell's P-N junction that may stem from imperfections in the crystal structure [10]. On the other hand, R_s may represent any connections between a cell and its neighbouring devices, including solder joints, junction box terminations, etc. [10]

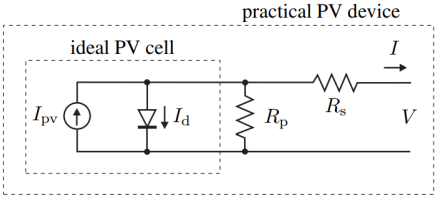


Figure 2: Single-Diode Model of PV Cell incl. Ideal Model [1]

Applying Kirchoff's Laws to the circuit, the output current, **I**, is modelled as

$$I = I_{pv} - \frac{V + IR_s}{R_p} - I_0 \left[e^{(\frac{V+iR_s}{V_t a})} - 1 \right], \tag{1}$$

The first term is the current from the cell, and the third term is the Shockley diode equation where $\mathbf{V_t}$ is the thermal voltage of the cells and **a** is the diode ideality constant. It is evident that the output current is the sum of the currents of the circuit, and the resulting I-V curve is a non-linear relationship as seen in figure 3, provided by [1]. On the I-V plot is shown additional points of interest: open circuit, short circuit and the maximum power point (MPP). The MPP is the point at which the PV cell produces the most power, where the product of current and voltage is optimised; this is visualised on the power curve also in figure 3.

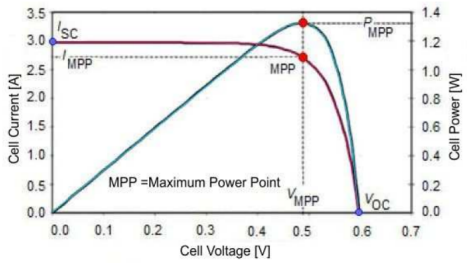


Figure 3: I-V Curve of Single-Diode Model with Points of Interest [2]

Although the SDM effectively captures the non-linearity of solar cell's behaviour, it may be difficult to compute in many situations. For example, a control system may benefit from operating on a linearised model to increase response times by reducing the computation power required. For this purpose, Thevenin and Norton equivalent circuits can be used to track the MPP when paramaterised for $\frac{dP_{pv,eq}}{dv_{pv,eq}} = 0$ [3]. Doing so will result in Norton and Thevenin equivalents, where:

$$I_{SC,N} = 2 \cdot I_{MPP}, \qquad R_{PV,N} = \frac{V_{MPP}}{I_{MPP}}$$

$$V_{OC,T} = 2 \cdot V_{MPP}, \qquad R_{PV,T} = \frac{V_{MPP}}{I_{MPP}}$$

So it can be seen in figure 4 that the equivalent circuits share the exact same MPP as the more accurate SDM.

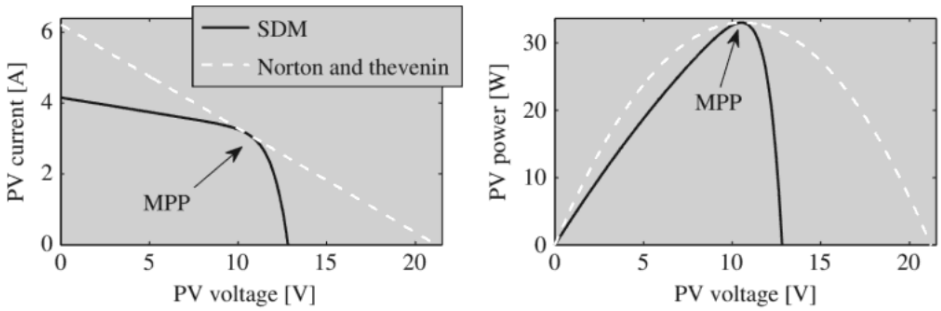


Figure 4: Comparison of SDM and Linearised Models [3]

Irradiance and Partial Shading

As previously mentioned, the cell produces a current that is proportional to the intensity of an incident light - this is commonly known as irradiance (specifically solar irradiation), and is a measure of power per area [W/m^2]. The irradiance a cell is exposed to will greatly vary its power output. [1] shows experimental data of a cell's IV curves at different irradiances using the Kyocera KC200GT at 25°C as seen in figure 5.

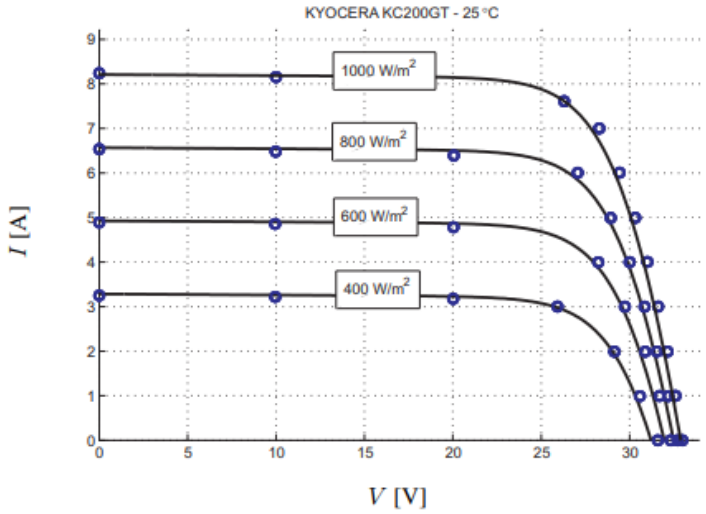


Figure 5: Experimental Data of A Cell at Varying Irradiations [1]

In many situations, a solar array will be installed on a flat surface for applications such as commercial or residential solar energy collection. Thus, the irradiance for each of the cells on a single panel will be equal. However, not all cells within a panel will have the same IV curves. This could be a result of physical damage, or local shading. When a cell is underperforming for whatever reason, it may limit the current of the string of cells to its current [11]; under certain conditions, the weaker cell(s) may be forced to operate under the higher current, where it becomes a load rather than a current source [11, 12].

A low cost solution is to add bypass diodes in parallel to strings. As seen in figure 6, the bypass diode provides an alternative route for conduction when one or more cells are performing sub-optimally: when a string is limiting current, the bypass diode will conduct due to the power mismatch between strings. Of course, this will mean that the module will lose all power produced from that string but it may be calculated that not using a bypass diode will cause a greater power loss or even cause damage to the cells from hotspots [11].

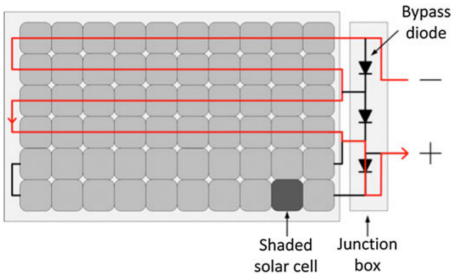


Figure 6: Diagram of Partially Shaded Solar Module with Bypass Diode [4]

The result of this is an IV curve which has multiple MPPs. In this case, there are two modes of operation (at MPP): The shaded cells are bypassed, or the current of the entire string can be limited to the shaded cells' current output so that it is not bypassed [5]. As shown in figure 7, there are multiple local maxima but still only one global maximum.

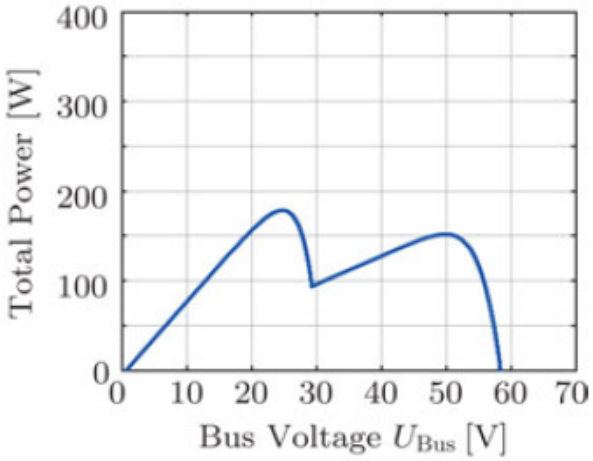


Figure 7: Example IV Curve of Partially Shaded Solar Module [5]

In addition to reducing power loss, bypass diodes are used to avoid damage to the cells, as operating the cells in reverse voltage breakdown can cause hotspots to form [13]. However, research conducted by [14] reveals that the SunPower Maxeon III cells can be used in large configurations with improved reliability; this is due to their low breakdown voltage. [14] also claims that operating the cells in breakdown causes no damage and does not affect performance.

2.2.1 Maximum Power Point Trackers (MPPT)

Ideally, the solar panel will always be operating at its MPP. However, environmental changes such as shading, obstructions, temperature and irradiance is constantly changing the characteristics of a solar array. To combat this, one may use a maximum power point tracker (MPPT) to automatically shift the operating point of the PV system so that it maintains its MPP. MPPs are realised as DC-DC converters of various topologies to achieve control of the PV system's operating point.

MPPT Architectures

The topic of MPPTs is an extensively researched field of power electronics, with some papers being published in as early as 1980 [15]. However, the conventional MPPT featuring a central converter/inverter (see fig. 8(a)) appears to be outdated nowadays. [7, 16, 5, 9, 6] are some more modern articles that present the results of analyses on multiple MPPT architectures. Developments in the field gave rise to distributed MPPT (DMPPT) architectures, where the MPP of cell groupings of various sizes can be tracked individually. Despite achieveing true MPPT, [7] shows that the two most dominant DMPPT methods still process 100% of the power, resulting in dependencies on the MPPT overall efficiency (see figure 8(b) and 8(c)).

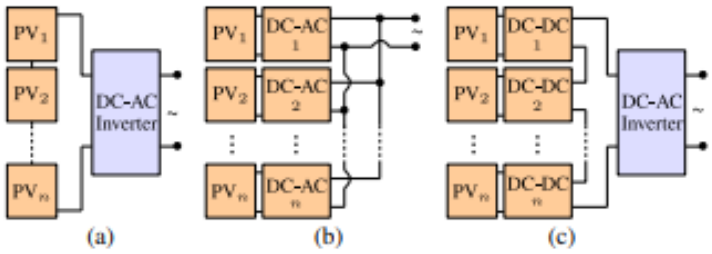


Figure 8: Various DMPPT Architectures [6]

Consequentially, many researchers ([17, 6, 7, 16]) have proposed different approaches that only process a fraction of the power. These architectures feature DC-DC converters that are connected to PV cells/strings in parallel rather than in series; a PV-PV example is as seen in figure in section 9.

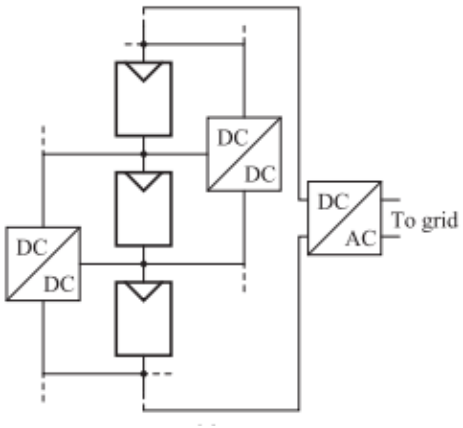


Figure 9: Various Partial Power Processing Architectures [7]

This sort of configuration has been labeled many things such as a shuffling converter [9] ("shuffles" power along the string), a differential power processor [16] or delta-converter [17] (only processes the power mismatch) and [sub]module-integrated-converter (subMIC) [5]. As expressed in [17], the main idea is to only enable the converters when a power differential occurs, so that the bulk of the power is still delivered by the PV devices. The result is the option to gain higher efficiency as shown in [7] or to reduce cost by using cheaper, less efficient converters. From an operational standpoint, this type of converter also has the advantage of forcing the PV system to behave as a single module with an average output power so that there is only one MPP to track, significantly reducing the required computational power.

2.2.2 MPP Tracking Techniques

Over the past few decades, countless MPPT techniques have been invented. To evaluate the mass literature on the topic would be a tremendous task on its own. Esram conducts an excellent literature review of over 90 papers and concisely summarises significant characteristics of each technique in a single table. This table can be found in Appendix **??**.
However, the Esram paper is only dated at 2007; while it is a great compilation of papers, it is still rather outdated. As a result, it may miss out on further research that is also highly relevant. [7] shows that achieving Voltage Equalization between PV strings results in a power output close to the MPP for minor mismatches. For applications where the mismatch is not significant, voltage equalisation can achieve tolerable results for a major simplification in control overhead.

2.3 Research Objectives

The end of the last semester witnessed the development of the shuffle converters from simulation to implementation in the solar vehicle. However, due to the tight timelines, limited amount of testing and data acquisition, reliable performance was not achieved and ultimately led to the failure of the devices. This led to a further discussion as to what the operational requirements should be for the shufflers, and also the methods used to determine such specifications.

2.3.1 Proposed Solution

The proposed direction following the events of the race was to analyse data from the PV system to determine the effects of both the shading and curvature on the array. From here, an informed testing regime would be devised; the shuffler converters can be observed operating in simulated or controlled environments.

The data to be used could have been collected from the central MPPT of the vehicle's PV system as a separate unit from the vehicle's chassis. Before doing so, the shuffler converters would have needed to be disconnected. Although the SunPower cells can operate in reverse breakdown with no effect on performance, it was preferable to connect bypass diodes to the array - there was a possibility that the processes of construction and competition may have caused physical damage to the cells, increasing risk of hotspots and other destructive phenomena. However, due to the unforeseen circumstances of the COVID-19 pandemic, the chance of in-person testing had greatly diminished. As a result, it was decided to steer the research task to a simulation-oriented direction.

2.3.2 Updated Proposal

While physical testing had been postponed, the MATLAB/Simulink models of the PV system were able to be further developed. During the development of the Simulink model for the shuffler converters in the initial stages of the project, it was seen that they were not capable of optimising the power output in all situations. It was observed that although the shufflers were able to achieve voltage equalisation, the cell module sizes had an effect on the optimisation ability of the shufflers.

The remainder of the project focuses around improving the PV system model. The first priority was to implement a traditional central P&O MPPT that represents the Elmar MPPT used in the vehicle. The P&O was to be developed in a series of models of increasing complexity. The end goal was to have a working model of the entire system. This will be of great benefit for a number of reasons. For example, the interactions between the shufflers and the Elmar can be observed and accounted for. In addition to this, firmware development times can be reduced by making use of the Parallel Computing Toolbox and also C code generation.

3 DESIGN

3.1 Methodology

To achieve the goal outlined in section 2.3.2, an iterative design approach was implemented. However with each design loop, the complexity of the design was increased. Figure 10 below describes the design flow of the solution. While the basic concept was based on a traditional iterative design loop, this chart effectively describes the multi-level aspect of this research task.

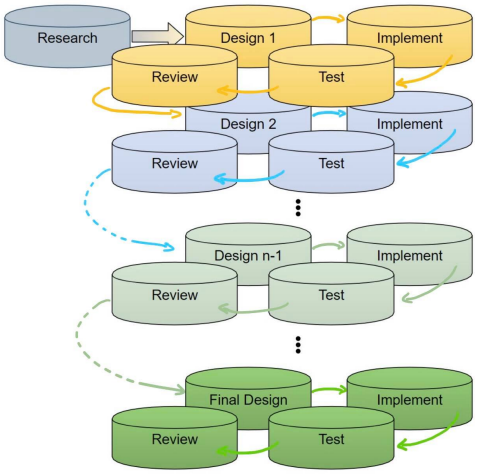


Figure 10: Flowchart of Design Methodology

Each level (or design n) represents a Simulink model of varying complexity. The aim was to incrementally build the MPPT model from fundamentals into the specific application. As such, a hierarchy of functionalities was established, of which the design structure is formed.

- Linearised PV model
- Non-linearised PV model
- Realistic PV array
- Array with shufflers

Depending on the depth of the model, various data sets for Irradiance inputs were used. For example, the linearised model only required a static irradiance to show Perturb and Observe functionality. However, the final design was verified using real irradiance data collected experimentally.

3.2 Simulation Tools

Given that the design process followed an extensive testing and development regime, it was of great value to be able to run multiple instances of simulations at once. Making use of `parsim()` in MATLAB's Parallel Computing Toolbox opened up opportunities to assign specific parameters a range of values. MATLAB was able to run these unique simulations in parallel, greatly reducing computation time.
Implementing `parsim()` required careful setup of the Simulink model to be compatible with a matching MATLAB script. Although tedious, the result was a powerful interface between MATLAB and Simlink, granting programmatic approaches to iterative design whilst using the complex mathematical models in Simulink.

3.3 Stage 1: Linearised Model

The first step taken to implement the MPPT was to build a very simplistic model that simulates the core functionality of the PV system. A screenshot of the model is shown below:

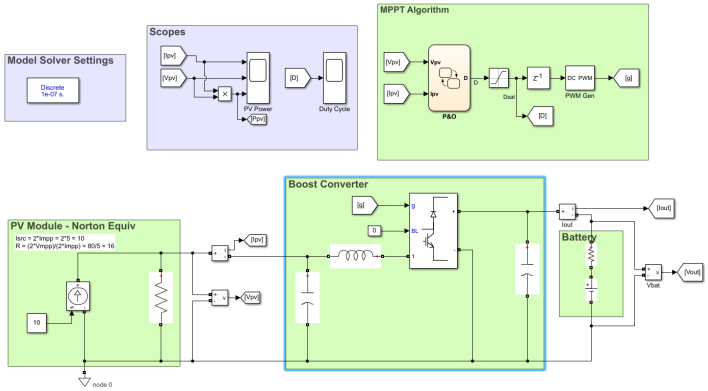


Figure 11: Simple Simulink Model of MPPT

As seen in figure 11, there are a number of subsystems that build the model. The functional subsystems, highlighted in green, feature the the PV module, boost converter, P&O algorithm, and battery.
The PV module is crudely modeled by its Norton equivalent according to [3]. Major advantages of this are significantly lower complexity, and an easily characterised MPP. In this example, the MPP will be set to be at $[V,I] = [80V, 5A]$. Thus, $I_{src} = 2 \cdot I_{MPP} = 10A$, and $R_{pv} = \frac{80}{5} = 16\Omega$. Knowing the actual MPP of the PV module will be incredibly useful in evaluating the performance of the MPPT.
The boost converter will remain largely unchanged throughout this iterative design process. The passive components of the converter will be as close to the original Elmar MPPT as

possible; this will be achieved by following the datasheet [18]. Namely, $C_{in} = C_{out} = 660\mu F$. While other values are not provided, assumptions will be made and 100% efficiency will be targeted so that the efficiency polynomial can be applied after the fact. To achieve a balance between accuracy and computation time, the *Switching function* model (model 2) is used. In this mode, the converter is controlled through a switched digital signal into the port <g>. This mode was chosen as it grants a greater level of control over the PWM signal's frequency and edge alignment. The PWM signal was generated by the PWM Generator Block, operating at 10kHz. However, in order to align the current and voltage sampling to the centre of the switching period, an Up-Down counter was used at twice the desired frequency. The result is a centre-aligned PWM signal.

The duty cycle signal which is fed into the PWM generator is calculated by the blocks in the MPPT Algorithm subsystem. These form the control loop based on the P&O algorithm in the form of a stateflow chart. [19] provides an excellent review of the functions offered by this component. In short, stateflow provides a graphical method of developing a finite-state machine. It offers three modes of operation: classic, Moore and Mealy. Classic stateflow charts combine functionality of both Moore and Mealy machines, but also adds extended semantics. However, for Moore and Mealy charts, system verification may be achieved easier due to the set rules of these machines. In addition to that, Moore machines can provide a smoother transition to hardware implementations. For the sake of speeding up development, a Classic Stateflow will be implemented; use of a Moore chart will be reconsidered in the future.

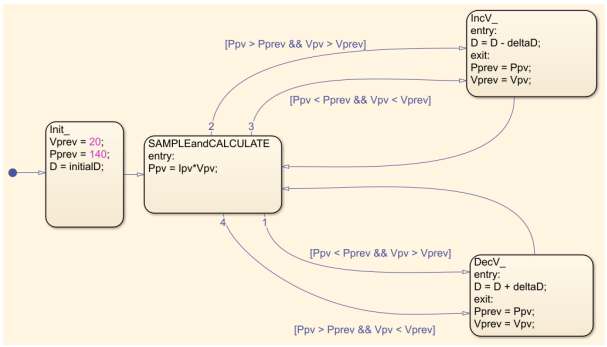


Figure 12: Classic Stateflow Chart in Simulink

Above is an image of the chart used in the stateflow block. It can be seen that functionality is very simple: after an initial state, the machine enters an intermediary state where the operating point of the PV module is evaluated. From here, the decision of which direction to perturb the voltage is made according to the P&O flowchart. The power point history is updated upon exit from the perturb states.

Results

A simple script was written to run a number of simulations with different parameters simultaneously, as shown below. The `for` loop stores the model and appropriate parameters in a Simulink object, with varying starting points for the duty cycle. The output states were stored in `out`. A simplified code snippet is included for brevity:

```
mdl = 'PnO_simple_model'; % Define which model to run

% Run a couple simulations with different parameters
for i = 1:6
    % Assign the model
    in(i) = Simulink.SimulationInput(mdl);

    % Assign constant and dynamic variables
    in(i) = in(i).setVariable('initialD',[0, 0.2, 0.4, 0.6, 0.8, 1]);
end
out = parsim(in); % Simulate models in parallel
```

In addition to logging Ppv, duty cycle, time, current and voltage are used for analysis. Running the simulation yields the following results:

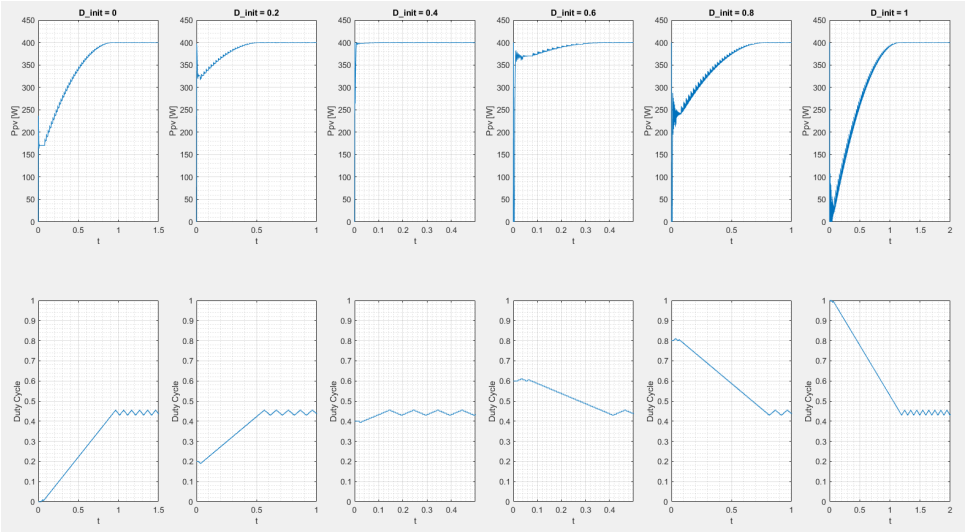


Figure 13: PV Power and Duty Cycle for Simple P&O Model

It can be seen that the MPPT manages to track and maintain the prescribed MPP of $80V \times 5A = 400W$ from all duty cycles. Inspecting the voltage and current signals individually reveals that it has indeed reached MPP.

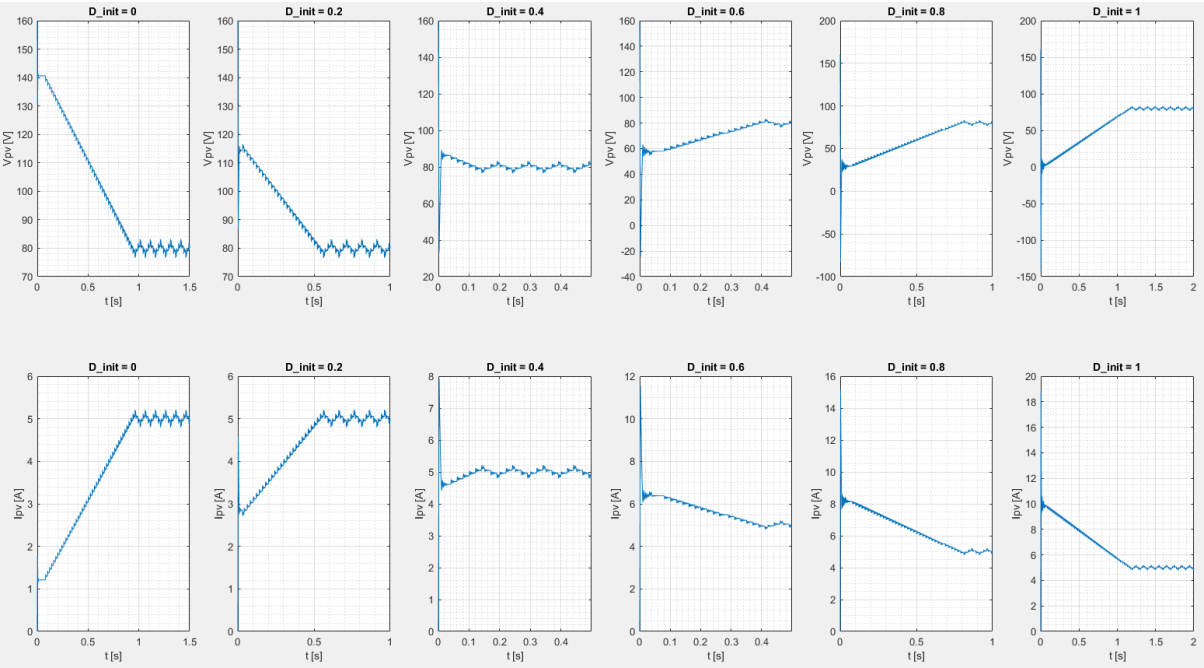


Figure 14: Voltage and Current for Simple P&O Model

In each simulation, the P&O eventually reaches a point where it oscillates between a duty cycle of 0.455 and 0.43. Of course this is a limitation of the barebones P&O algorithm that moves in fixed steps. The time it takes to reach the MPP is dependent on the distance between the initial and final values. Reducing the oscillation amount can be achieved by decreasing the perturbation step size, however this will increase the response time. Witnessing this behaviour is merely evidence that the tracking algorithm is functioning as intended.

3.4 Stage 2: Non-linearised PV Module

After the stateflow P&O block was deemed effective, it was implemented into a simulation with a more accurate PV model. The SDM was be implemented through the use of the *PV Array* block. This block models an array of definable PV modules using a five-parameter model based on a current source, diode, and shunt and series resistors to define the irradiance and temperature based characteristics. These equivalent circuit parameters are calculated using operating characteristics input by the user, such as V_{oc}, I_{sc}, V_{mpp}, I_{mpp}, and the temperature coefficients for both V_{oc} and I_{sc} as well as array wiring configurations. When using the PV Array block, the behaviour is modeled according to irradiance and temperature inputs. Below is shown the I-V curve generated by Simulink; the array is subjected to $1000W/m_2$ at different temperatures.

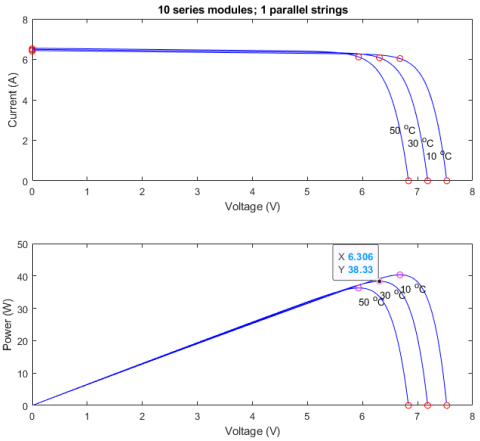


Figure 15: I-V Curve Generated By Simulink

Marked on the P-V curve is the maximum power output of one string of 10 modules in series, which is 38.33W. The non-linearised PV Simulink model will use 6 of these blocks in series to emulate one sub-array of the solar vehicle. The full model (see figure 16) features 6 PV blocks as described, along with a boost converter and the P&O Stateflow chart. In this phase, the array was treated as if it were flat, so each block was fed equal irradiance values. The total theoretical maximum power output of the PV system is $38.33 \times 6 = 229.98W$. The only significant change from that in section 3.3 is the introduction of the PV blocks.

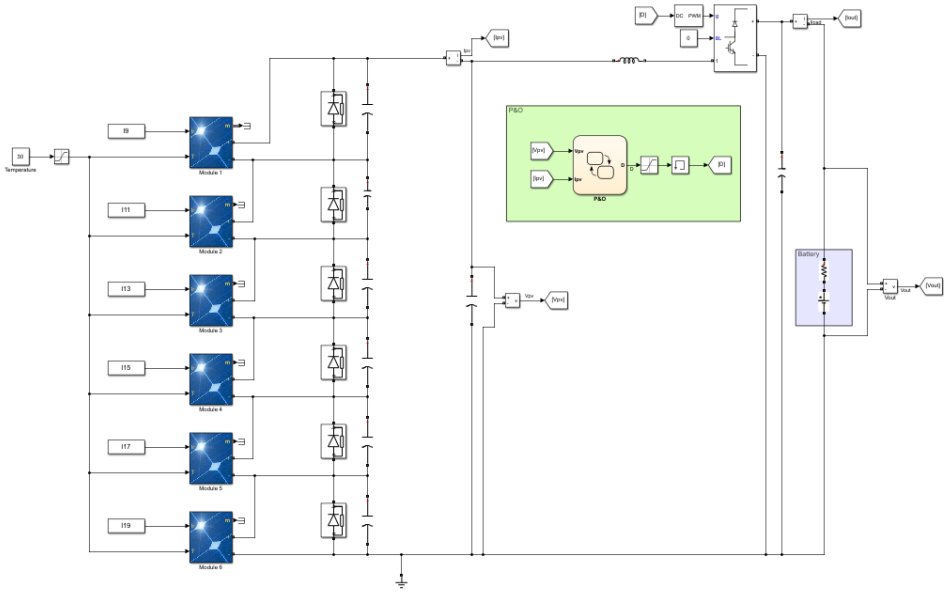


Figure 16: Simulink Model with PV Array Blocks

As an initial test, a static value was fed into the irradiance input for equal sized strings simulating a flat array (equal irradiance for all strings). Fig. 17 shows the results from a simulation at $1000 W/m^2$. It's observed that the MPPT successfully converges toward MPP from a range of initial operating points. Each starting point showed its ability to provide around 223W which is within 3% of the theoretical array MPP. However, the right-most case shows significant oscillation resulting in inconsistent power output.

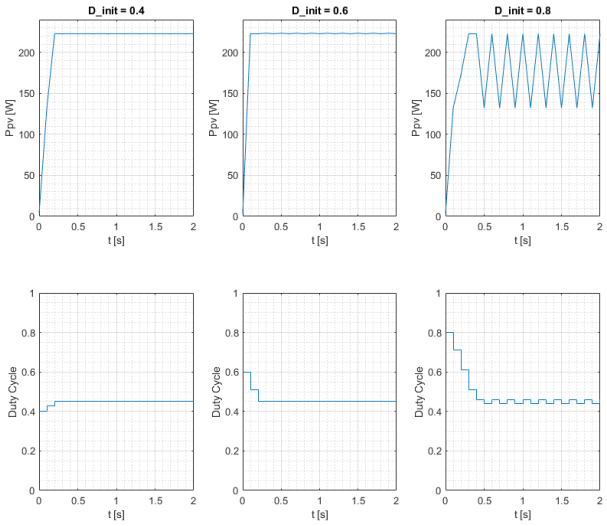


Figure 17: Static Irradiance Tracking

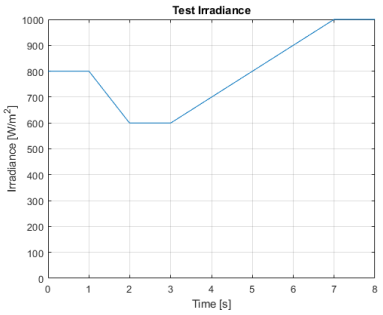


Figure 18: Plot of Test Irradiance

For the next test, some dummy irradiance data was created to check the dynamic tracking capabilities of the P&O tracker. Figure 18 is a plot of the constructed data. The major features of this data set are the three plateaus of a known MPP connected by gradients. The plateaus are at 800W/m^2, 600W/m^2 and 1000W/m^2 with MPPs of approximately 185W, 137.5W, and 233.5W respectively.

Below are the results of the nonlinearised model performing under simulated irradiances. It can be observed that the duty cycle requires an amount of time proportional to the distance away from the optimal duty cycle. However, when optimal power is detected, it tracks MPP fairly well. The first case on the left shows the plateaus to be approximately close to the expected values, and the same for the other cases when MPP is reached.

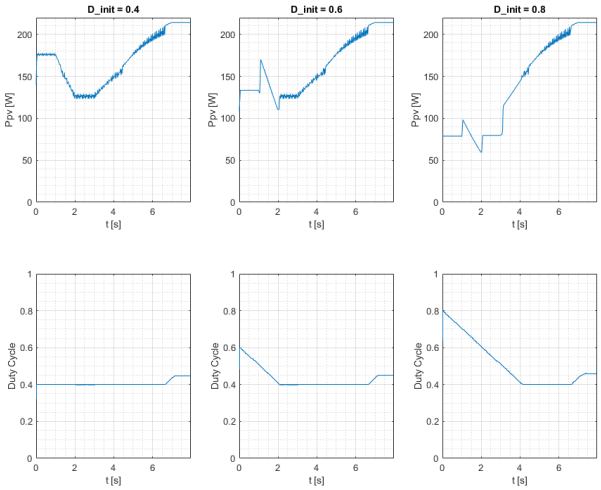


Figure 19: Results of NL Model With Test Irradiance

The visible noise is due to the switching behaviour of the boost converter. When inspected closely, the ripple was very characteristic of the charging and discharging. The noise had been filtered out to demonstrate the tracker's ability more effectively.

3.5 Stage 3: Shuffle Converters

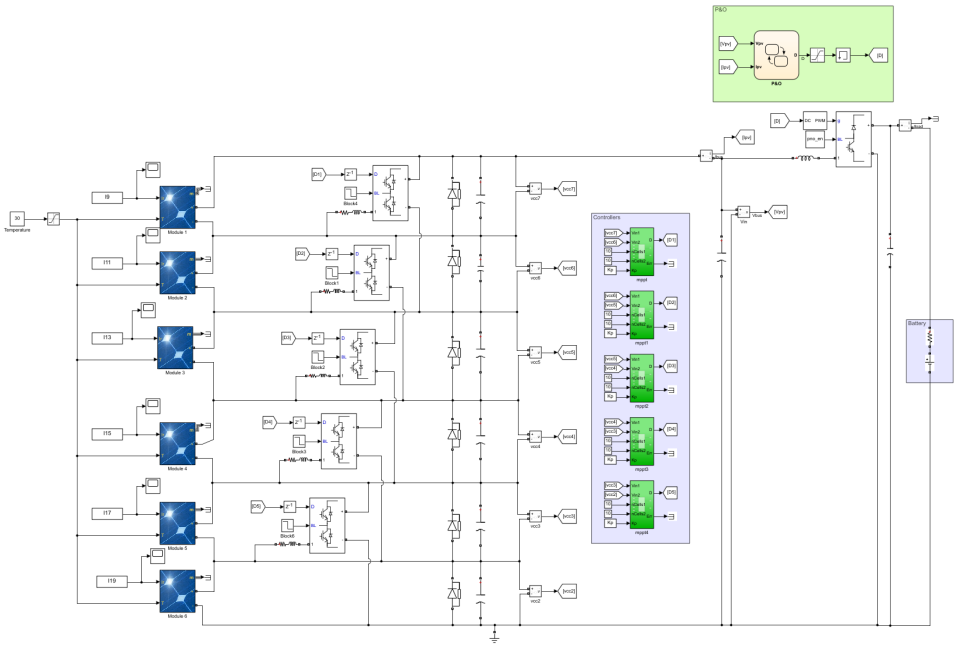


Figure 20: Simulink Model of PV System With Shuffle Converters

Figure 20 shows the Simulink model used to test the shuffle converters with the P&O MPPT. The same base PV model was used for consistency, and the shuffle converters were implemented using the relevant blocks for the half bridges and controllers. The controllers were implemented using a MATLAB code block for each of the shufflers. The code blocks featured a slightly updated version of the algorithm used throughout the first section of this project: a P controller with the aim of achieving voltage equalisation. The algorithm is summarised as:

```
vcell1 = vin1/num_cells1; % Normalise measured voltages to V/cell
vcell2 = vin2/num_cells2;

err    = vcell2 - vcell1; % Calculate error

D      = vcell1/(vcell1+vcell2) + Kp*err; % Calculate new duty
```

Since both the centralised and distributed MPPTs were both tested in isolation, much of the focus was placed on optimising the integrated system for operation in the real simulated cases. The data used was gathered to estimate the available solar power on the route of the Solar Challenge. Figure 21 shows the irradiance as perceived by the modules in the 3rd string for the first couple kilometers.

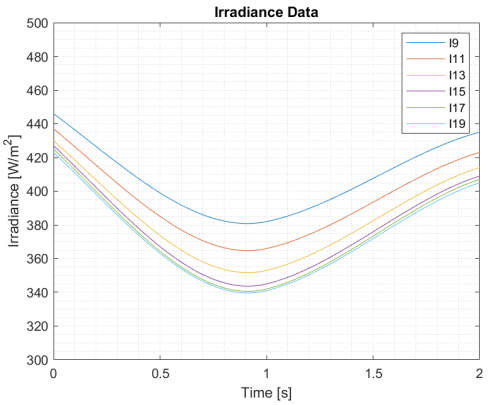


Figure 21: Sampled Irradiance Data

The data was 'sped up' to offer more variety; the more dynamic behaviour can be applied to other situations that have cloud coverage or additional obstructions casting shadows. To speed up the data, the "kilometers travelled" were converted to metres and then scaled to fit the speed of 1000m/s. A vector was then interpolated to match the simulation time vector using `interp1()`. This was repeated for the irradiances for the relevant modules.

Testing in Isolation

To show that the shuffle converters were capable of achieving voltage equalisation, they were tested in isolation from the Elmar MPPT. Inputting the irradiance data for the 3rd string revealed that the controller effectively equalised the voltages of the modules subjected to differing irradiance values.

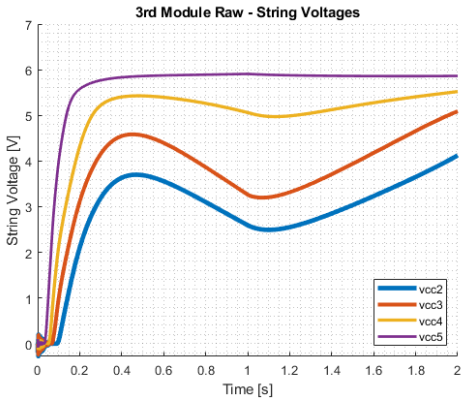


Figure 22: String Voltages of 3rd Module: Raw

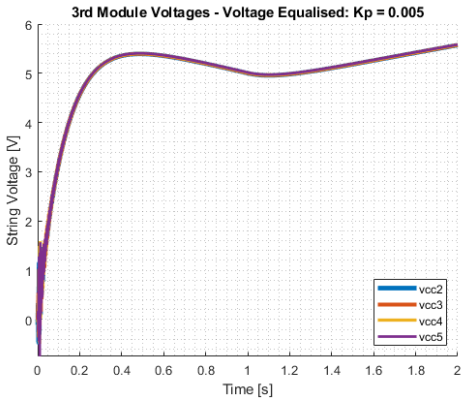


Figure 23: String Voltages of 3rd Module: Equalised

Figure 22 shows a significant voltage difference between vcc2 and vcc4 or vcc5, yet the shufflers are perfectly capable of equalising the cell voltages of each of the modules. Plotting the power output of the PV system revealed that the shufflers increased the output by up to 3W instantaneously.

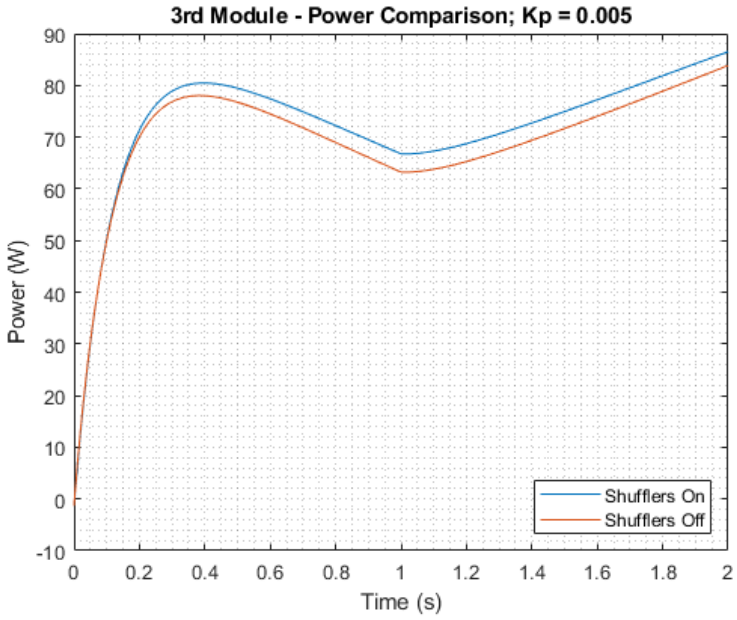


Figure 24: Power Output Comparison of 3rd Module

Reintroducing The Elmar MPPT

After verifying the voltage equalisation operates as intended, the entire system was finally simulated as one. The result, show in figure 25 below, is obviously very noisy.

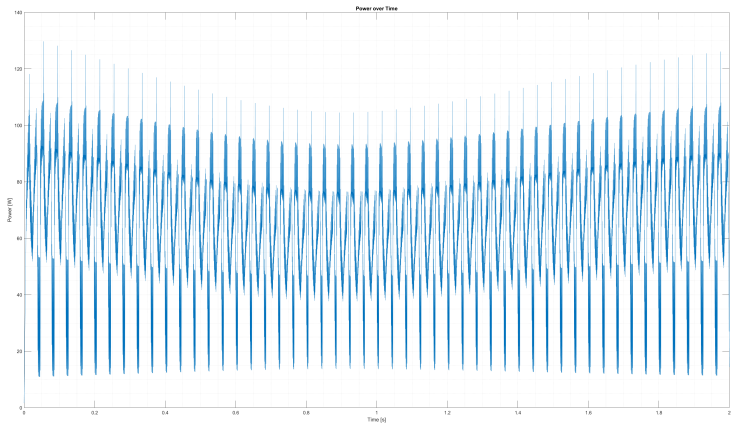


Figure 25: Power over Time For Full Model

However, zooming into the periodic noise, it is clear that what is observed is a combination of the 'low' frequency switching of the Elmar MPPT and the higher frequency switching of

the shuffle converters. The overall shape of the signal is very typical of a boost converter, and similar behaviour can be interpreted in the higher frequency ripples. However, resolution is lost due to the sampling rate of the simulation. When the module voltages are observed, similar behaviour is exhibited due to the operation of the bridges that form the shuffle converters. Of course, the signals appear as one because they are all equal due to the shuffle converters.

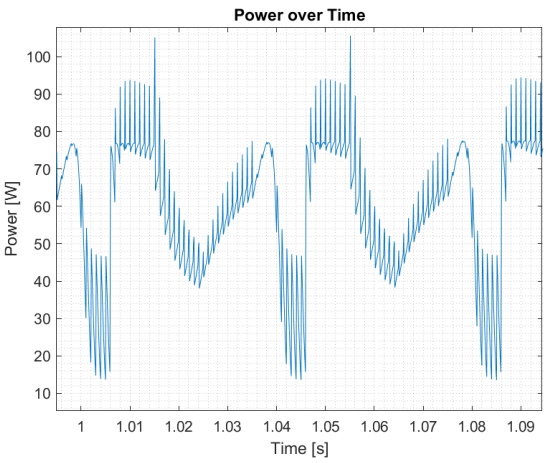


Figure 26: Closeup of Full Model Power

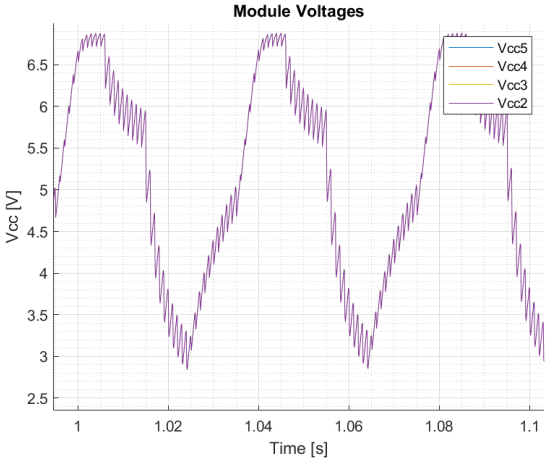


Figure 27: Closeup of Individual Voltages in FM

To draw a quantitative comparison between using shuffle converters and not, the energy produced over the test period can be measured. This was achieved through the MATLAB function `trapz(X, Y)`, which performs numerical integration of vector Y over vector X. In this context, the result will be in Joules (or Watt-seconds). The total energy produced by in the 2 seconds by the system without shuffle converters is **119.9907J**, whereas the energy produced by the system with shuffle converters in the same scenario is **125.8395J**. This results in about a 1.05% increase in efficiency.

3.6 Summary

Through various stages of model complexity, the functionality and effectiveness of both distributed and central MPPTs were demonstrated. In the last section, these two technologies were verified to be compatible and offer a deeper level of MPP tracking when combined. The shuffle converters proved to be effective at offering "pre-maximisation" for the centralised MPPT.

4 PROJECT TIMELINE

The Gantt chart below shows approximate progress development for the past semester.

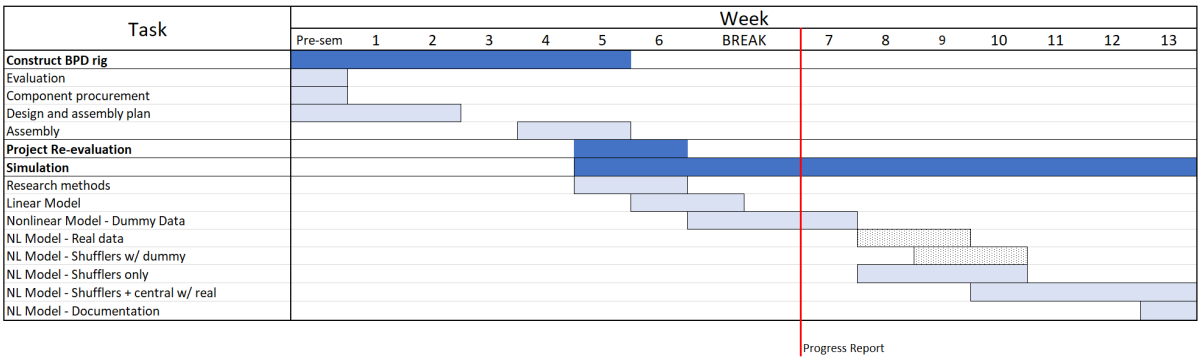


Figure 28: Gantt Chart of Project

In comparison to the projected timeline at the time of the mid-semester report, development had progressed similarly to what was expected. The project plan was adjusted according to the requirements of the design. Not only did this involve creating additional intermediate steps, but also removing steps that were not so necessary. The steps omitted are signified by the grey dotting. Having said this, some features took longer than expected to implement others not so much, hence the occasional omission. However, it was correct to assume the final stages of the project would consume more time and resources due to the nature of increasing complexity.

5 CONCLUSION AND RECOMMENDATIONS

This research task began as a project to develop and implement distributed MPPTs to further increase the performance of a vehicle's PV system. While previous work had been able to assist with developing the foundation of the devices, much of the first portion of this project in particular was a scramble to implement the hardware under unforgiving time restrictions. As a result, the first half of the project saw an underdeveloped product being installed which led to its ultimate failure. After careful consideration, it was decided to reallocate resources into research and testing to ensure the product operates safely and reliably. This project had sought to bring forth the fundamentals required for deeper research and testing regimes that could later be applied to a multitude of solar configurations.

Following the work of a previous student researcher, this paper was able to successfully establish a research platform to efficiently develop MPPT algorithms tailored to automotive applications. This was made possible by the powerful models provided by Simulink, and its integration with MATLAB's Parallel Computing Toolbox to enable rapid iterative development while applying programmatic practices. In doing so, a number of Simulink models and accompanying testing scripts were developed, which have the potential to streamline the development of various custom PV systems.

The findings of this report have demonstrated that a simple voltage-equalising DPP MPPT algorithm was able to maximise the power output of the automotive's PV system. However, the results are more or less specific to the test conditions explored as the algorithms used require tuning (perturbation increments, control values, timing requirements, etc). Before the shuffle converters can be confidently deployed, further research must be conducted to characterise the operational boundaries.

Conducting in situ testing with the PV array will provide the required data to safely determine the hardware requirements via simulation, i.e. potential shading circumstances, current limits, shut-off time, etc. This may require specific instrumentation such as a faux bypass-diode device that measures when a diode conducts, or a testing rig that is able to selectively control the irradiance on the array's surface. Constant feedback between simulation and experimental data would be of tremendous value. With these methods, it would be possible to further optimise for the route of the World Solar Challenge or generate enough data for the MPPTs to be a viable product.

The end goal of all these steps is to develop an algorithm that achieves true MPP tracking for strings of differently sized modules. When solar technology becomes developed enough, shuffle converters will no doubt play a role in bringing sustainable transport to the world.

REFERENCES

[1] Marcelo Gradella Villalva, Jonas Rafael Gazoli, and Ernesto Ruppert Filho. Modeling and circuit-based simulation of photovoltaic arrays. In *2009 Brazilian Power Electronics Conference*, pages 1244–1254. IEEE, 2009.

[2] JA Ramos-Hernanz, JJ Campayo, E Zulueta, O Barambones, P Eguía, and I Zamora. Obtaining the characteristics curves of a photocell by different methods. In *International Conference on Renewable Energies and Power Quality*, volume 11, pages 1–6, 2013.

[3] Giovanni Petrone, Carlos Andrés. Ramos-Paja, and Giovanni Spagnuolo. *Photovoltaic sources modeling*. John Wiley Sons, first edition, 2017.

[4] Boudewijn B Pannebakker, Arjen C de Waal, and Wilfried GJHM van Sark. Photovoltaics in the shade: one bypass diode per solar cell revisited. *Progress in photovoltaics: Research and Applications*, 25(10):836–849, 2017.

[5] Matthias Kasper, Dominik Bortis, and Johann W Kolar. Classification and comparative evaluation of pv panel-integrated dc–dc converter concepts. *IEEE Transactions on Power Electronics*, 29(5):2511–2526, 2013.

[6] Carlos Olalla, Christopher Deline, Daniel Clement, Yoash Levron, Miguel Rodriguez, and Dragan Maksimovic. Performance of power-limited differential power processing architectures in mismatched pv systems. *IEEE Transactions on Power Electronics*, 30(2):618–631, 2014.

[7] Shibin Qin, Christopher B Barth, and Robert CN Pilawa-Podgurski. Enhancing microinverter energy capture with submodule differential power processing. *IEEE Transactions on Power Electronics*, 31(5):3575–3585, 2015.

[8] World Solar Challenge. 2019 regulations. Available at `https://www.worldsolarchallenge.org/files/2272_2019_bwsc_regulations_release_version_3.pdf`.

[9] Geoffrey R Walker, John Xue, and Paul Sernia. Pv string per-module maximum power point enabling converters. 2003.

[10] Faisal Ghani, G Rosengarten, and M Duke. The characterisation of crystalline silicon photovoltaic devices using the manufacturer supplied data. *Solar Energy*, 132:15–24, 2016.

[11] Vítězslav Benda. Photovoltaics: The basics. In *A Comprehensive Guide to Solar Energy Systems*, pages 151–179. Elsevier, 2018.

[12] Ali M Eltamaly. Performance of mppt techniques of photovoltaic systems under normal and partial shading conditions. In *Advances in renewable energies and power technologies*, pages 115–161. Elsevier, 2018.

[13] Jan Bauer, Jan-Martin Wagner, Andriy Lotnyk, H. Blumtritt, N. Zakharov, and O. Breitenstein. Physical mechanisms of electrical breakdown in silicon solar cells. *Annual report of Max Planck Institute of Microstructure Physics*, pages 44–45, 10 2009.

[14] D. D. Smith, P. J. Cousins, A. Masad, S. Westerberg, M. Defensor, R. Ilaw, T. Dennis, R. Daquin, N. Bergstrom, A. Leygo, X. Zhu, B. Meyers, B. Bourne, M. Shields, and D. Rose. Sunpower's maxeon gen iii solar cell: High efficiency and energy yield. In *2013 IEEE 39th Photovoltaic Specialists Conference (PVSC)*, pages 0908–0913, 2013.

[15] LL Bucciarelli, BL Grossman, EF Lyon, and NE Rasmussen. Energy balance associated with the use of a maximum power tracer in a 100-kw-peak power system. Technical report, Massachusetts Inst. of Tech., Lexington (USA). Lincoln Lab., 1980.

[16] Shibin Qin, Stanton T Cady, Alejandro D Dominguez-Garcia, and Robert Carl Nikolai Pilawa-Podgurski. A distributed approach to maximum power point tracking for photovoltaic submodule differential power processing. *IEEE Transactions on Power Electronics*, 30(4):2024–2040, 2014.

[17] Henk Jan Bergveld, Dick Büthker, Cristiano Castello, Toby Doorn, Adrie de Jong, Ralf van Otten, and Klaas de Waal. Module-level dc/dc conversion for photovoltaic systems: The delta-conversion concept. *IEEE Transactions on Power Electronics*, 28(4):2005–2013, 2012.

[18] Elmar Solar. *Elmar Solar MPPT Datasheet*, 3 2019.

[19] Rana Ahmed, A. Namaane, and Nacer M'Sirdi. Improvement in perturb and observe method using state flow approach. *Energy Procedia*, 42:614–623, 12 2013.

[20] Trishan Esram and Patrick L Chapman. Comparison of photovoltaic array maximum power point tracking techniques. *IEEE Transactions on energy conversion*, 22(2):439–449, 2007.

[21] Sathish Kumar Kollimalla and Mahesh Kumar Mishra. A new adaptive p&o mppt algorithm based on fscc method for photovoltaic system. In *2013 International Conference on Circuits, Power and Computing Technologies (ICCPCT)*, pages 406–411. IEEE, 2013.

[22] Abdelhamid Loukriz, Mourad Haddadi, and Sabir Messalti. Simulation and experimental design of a new advanced variable step size incremental conductance mppt algorithm for pv systems. *ISA transactions*, 62:30–38, 2016.

A APPENDICES

A.1 Cell Groupings of Solar Array

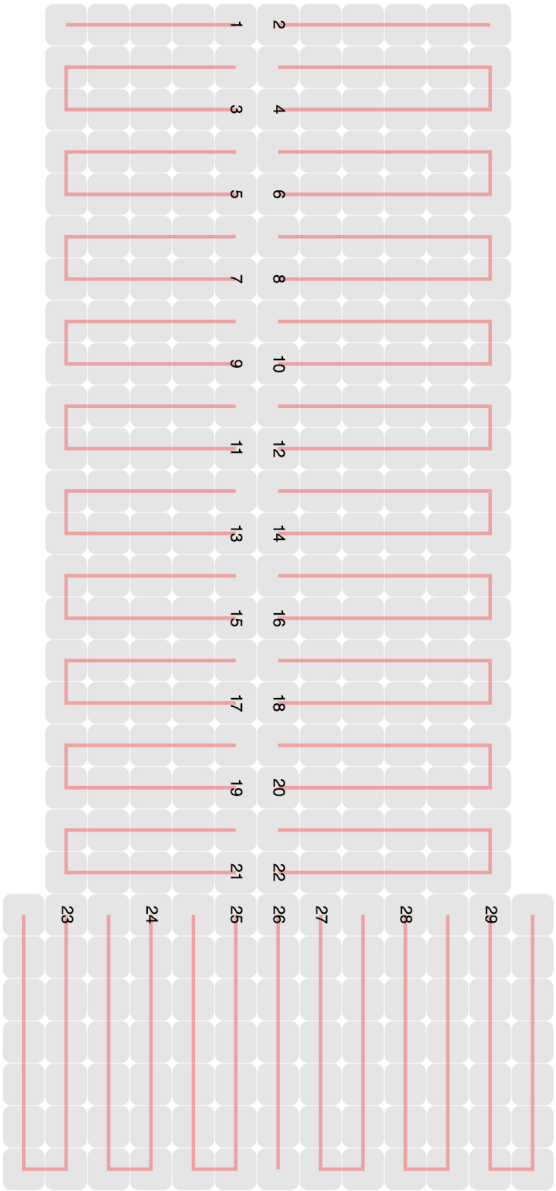


Figure 29: Module Positions on Array

A.2 Sub-division of Solar Array

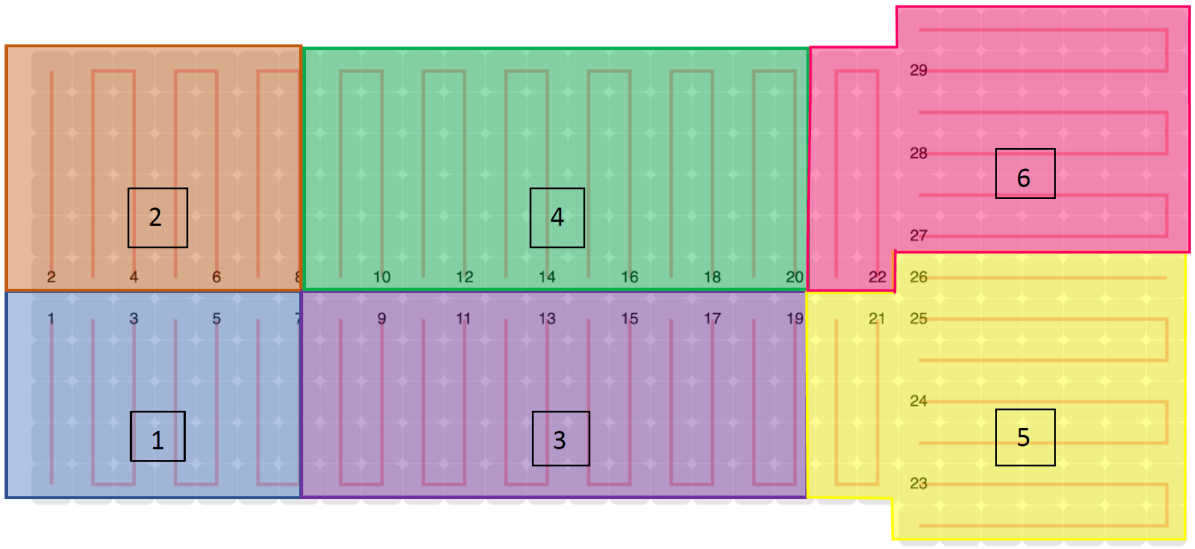


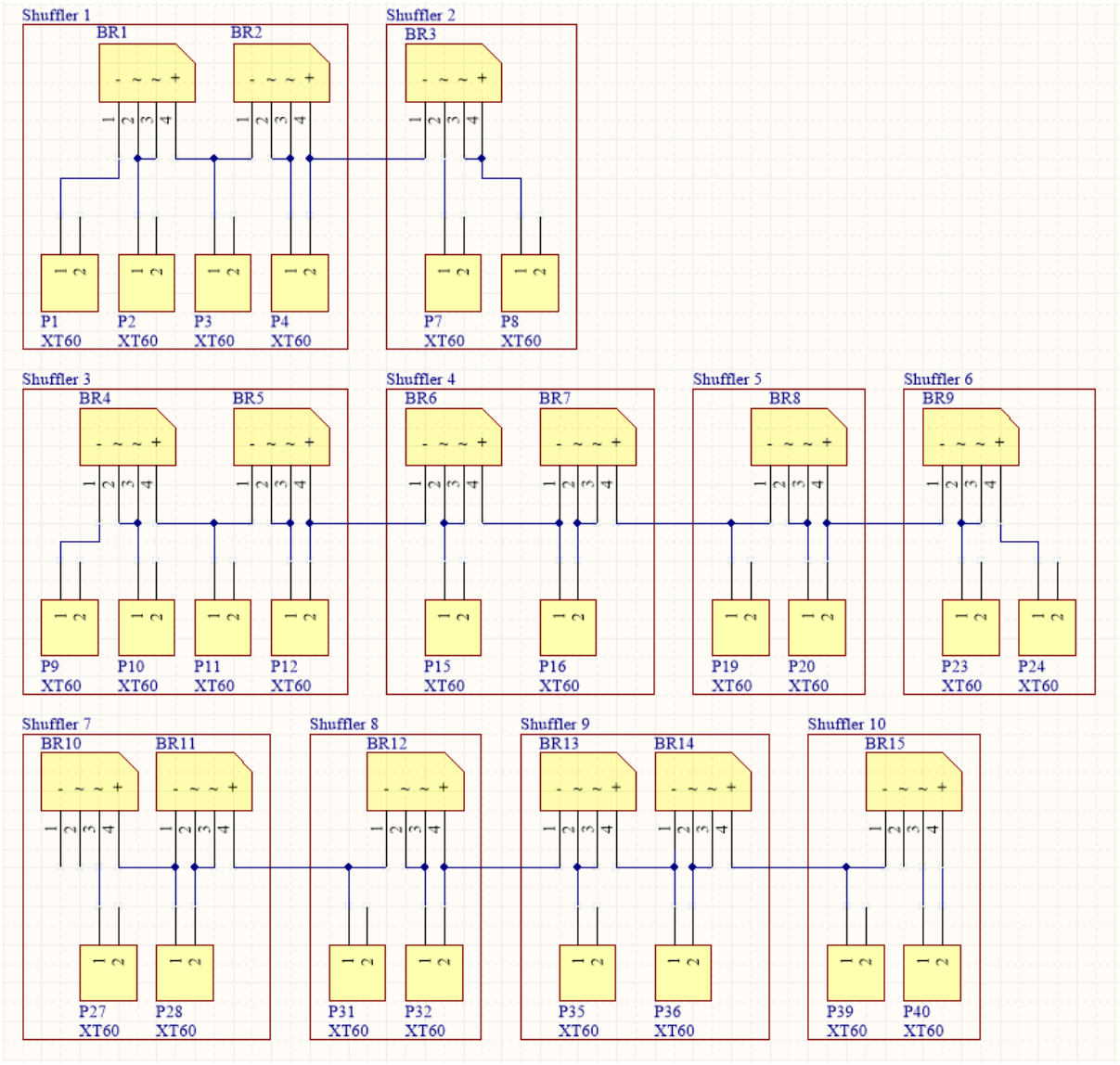
Figure 30: Subdivision of Arrays into Strings

A.3 Cell Count of Modules

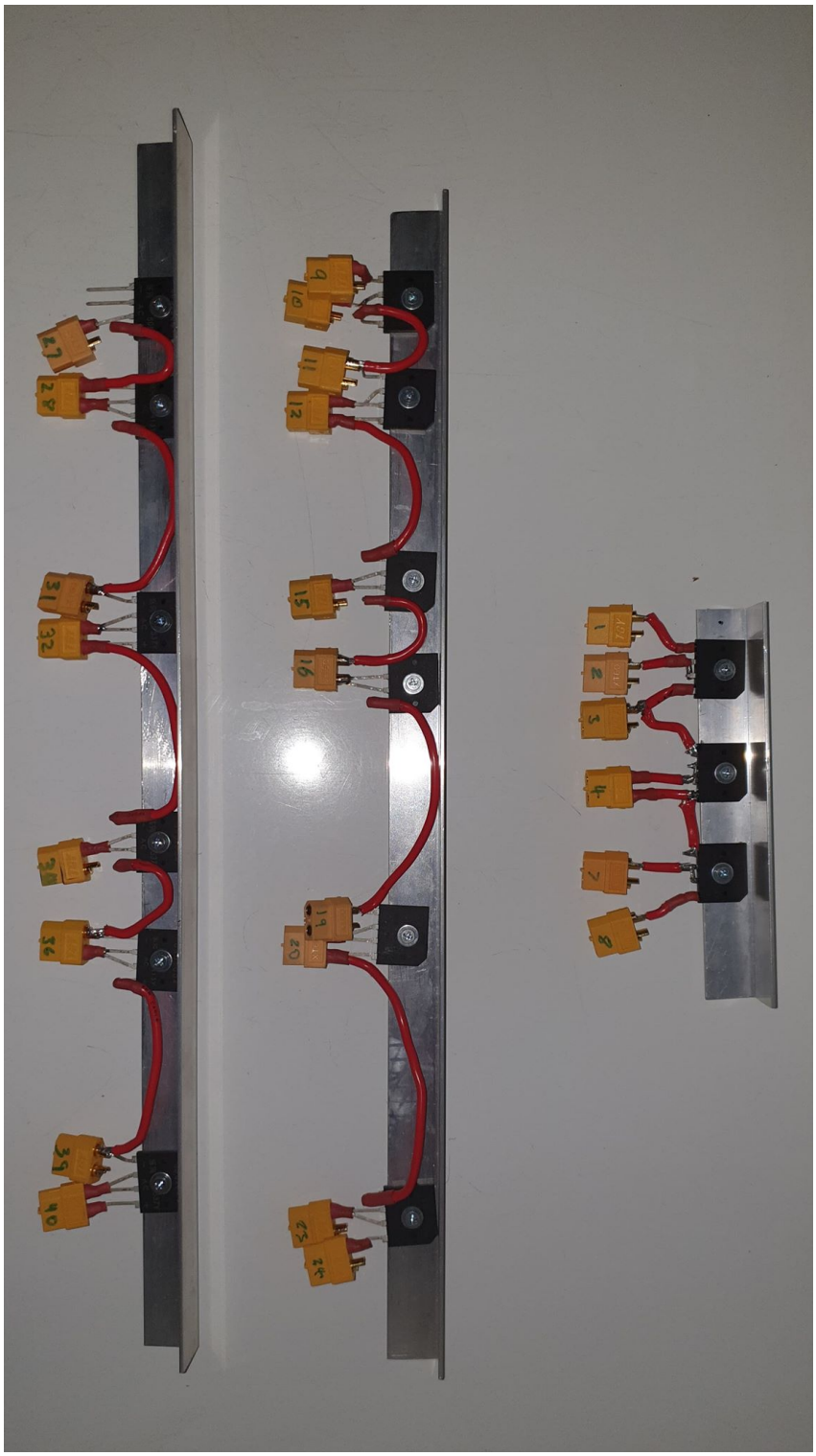
Module ID	Number of Cells	String Number
1	5	1
2	6	2
3	10	1
4	12	2
5	10	1
6	12	2
7	10	1
8	12	2
9	10	3
10	12	4
11	10	3
12	12	4
13	10	3
14	12	4
15	10	3
16	12	4
17	10	3
18	12	4
19	10	3
20	12	4
21	10	5
22	12	6
23	14	5
24	14	5
25	14	5
26	7	5
27	14	6
28	14	6
29	14	6

Table 1: PV Cell Assignments

A.4 Schematic Diagram of Bypass Diode Rig



A.5 Photograph of Bypass Diodes



A FINAL MATLAB INTERFACE SCRIPT FOR SIMULINK

Contents

- Centralised MPPT Batch Test
- About the models
- Load some data in
- Preparation
- Process irradiance data
- Run the sims
- Filter the data
- Plot some stuff
- Plot Irradiance
- Shuffle converter algorithm
- Remap data

Centralised MPPT Batch Test

This script was made to do multiple runs of a PV sim, specifically testing the performance of a centralised MPPT based on the P&O algorithm, using a stateflow block. This work has been done to provide a more accurate model of the PV system on a solar vehicle where the panels are part of a curved surface. However it can be applied to any system by adjusting the irradiance data, PV config, component values and control timings (i.e. switch rate, P&O rate, etc). At first, voltage equalisation between the modules may be achieved but after that, true MPPT should be the goal.

This script utilises the Parallel Computing Toolbox to run the Simulink simulations in parallel. Currently, it is only configured to use the CPU cores to run a simulation rather than the GPU. GPU computing was deemed less effective after using the builtin benchmark tool: 'paralleldemo_gpu_benchmark' Anyone wishing to run this script should have the Parallel Computing Toolbox installed, or can otherwise simply change parsim for sim to run the simulations serially.

Have fun :) Nathan Hui

```
warning off MATLAB:legend:IgnoringExtraEntries;

clear in out testVals
```

About the models

This script has been optimised for use with some simulink models developed as part of the project.

```
%
% The first model is pno_shufflers_3rdstring.slx which is an implementation
% of the shuffle converters in conjunction with the Centralised MPPT (that
% represents the Elmar MPPT).
% The model is an extension of 'pno_only_3rdstring' in which the Elmar is
% tested in isolation from the shuffle converters on the same PV system.
% This /should/ be identical to pno_shufflers_3rdstring except for
% the lack of shuffle converters Not all of the parameters are required for this model
% include the shuffle converters
% 'pnosimple' is an implementation of the P&O stateflow using a Norton
% equivalent circuit for the PV cell. Easily determined MPP, but lacks the
% nonlinear characteristics of the cells.
%
% Choose model to run

mdl = 'pno_shufflers_3rdstring';
% mdl = 'pno_only_3rdstring';
% mdl = 'pnosimple';
```

Load some data in

The irradiance data should be loaded in from 'extracteddata.mat' (just a workspace saved from a script extracting the irr data from VIdataS1)

You may find it's a bit clunky to deal with discrete arrays but it has its benefits. Feel free to combine into one container.

```
load('extracteddata.mat');
```

Preparation

```
% Set some parameters for the PnO stateflow
Ptol     = 0;         % Tolerance for PV power change
```

```
Vtol     = 0;         % Tolerance for PV voltage change
initialD = 0.1;       % Starting value for D
deltaD   = 0.001;     % Duty perturbation
pno_en   = 0;         % Enable pin for pno (inverted)
f_pno    = 100;       % Frequency of PnO (Hz)
t_pno    = 1/f_pno;   % Time period of PnO
f_pwm    = 2000;      % Frequency of PWM (using up-down counter) (Hz)
t_pwm    = 1/f_pwm;   % Time period of PWM

% Set some parameters for the shuffle converters
f_shf  = 1000;        % Shuffle frequency (Hz)
t_shf  = 1/f_shf;     % Time period of shuffle loop
f_spwm = 100000;      % Frequency of shuffle pwm (Hz)
t_spwm = 1/f_spwm;    % Time period of shuffle pwm
en     = 0;           % Enable pin for shufflers (inverted logic)
Kp     = 1;           % Proportional gain for shufflers

% Choose some simulation parameters
f_samp   = 10000;     % Sampling frequency (Hz)
t_samp   = 1/f_samp;  % Sample time for recording data

t_sim    = 2;         % Time of simulation in seconds
dt       = 0.01;      % Time step in seconds
n_samps  = t_sim/dt;  % Number of samples
time     = linspace(0, t_sim, n_samps); % Create a time vector

% Define the parameter array to be tested
n_runs   = 6; % number of runs
testVals = linspace(0, 1, n_runs+1); % Test parameter sweep
testVals = testVals(1:end); % Narrow down selection
testVals = [0];           % Or enter test values directly
n_runs   = length(testVals); % redefine n_runs if necessary
```

Process irradiance data

```
% Change the value of v to choose the 'simulated speed' i.e. the car moves
```

```
% at 20.83m/s = 75kph to see real time or set to 1000m/s to model at 1km
% every simulation second
v = 1000; % m/s
start = 0; % Move the starting position along the data set
I9 = map2time(I9, time, v, start);
I11 = map2time(I11, time, v, start);
I13 = map2time(I13, time, v, start);
I15 = map2time(I15, time, v, start);
I17 = map2time(I17, time, v, start);
I19 = map2time(I19, time, v, start);

%{
These  are the theoretical MPPs of some notable points
Pmpp = [1000, 233.46;
        800, 185.1;
        600, 137.52;
        400, 90.36];
%}
```

Run the sims

The array testVals can be substituted into any parameter desired

```
for i = length(testVals):-1:1
in(i) = Simulink.SimulationInput(mdl);

% Simulation parameters
in(i) = in(i).setVariable('t_sim',t_sim);
in(i) = in(i).setVariable('t_samp',t_samp);

% Test conditions
in(i) = in(i).setVariable('I9',I9);
in(i) = in(i).setVariable('I11',I11);
in(i) = in(i).setVariable('I13',I13);
in(i) = in(i).setVariable('I15',I15);
in(i) = in(i).setVariable('I17',I17);
in(i) = in(i).setVariable('I19',I19);
```

```
% Pno parameters
in(i) = in(i).setVariable('Ptol',Ptol);
in(i) = in(i).setVariable('Vtol',Vtol);
in(i) = in(i).setVariable('initialD',initialD);
in(i) = in(i).setVariable('deltaD',deltaD);
in(i) = in(i).setVariable('pno_en',pno_en);
in(i) = in(i).setVariable('t_pno',t_pno);
in(i) = in(i).setVariable('t_pwm',t_pwm);

% Shuffle parameters
in(i) = in(i).setVariable('Kp',Kp);
in(i) = in(i).setVariable('en',en);
in(i) = in(i).setVariable('t_shf',t_shf);
in(i) = in(i).setVariable('t_spwm',t_spwm);

end

% Do sims all at once
tic;
out = parsim(in, 'ShowProgress', 'on', 'ShowSimulationManager', 'on')'
toc;
```

Filter the data

This section creates an nth average lowpass filter Like downsampling, but better

n = 100; for i = n_runs:-1:1 out(i).t = downsample(out(i).t(1:end-1),n)'; out(i).Ppv = mean(reshape(out(i).Ppv(1:end-1),n,[]))'; out(i).Ipv = mean(reshape(out(i).Ipv(1:end-1),n,[]))'; out(i).Vpv = mean(reshape(out(i).Vpv(1:end-1),n,[]))'; out(i).D = mean(reshape(out(i).D(1:end-1),n,[]))'; end

Plot some stuff

```
% Plot the Module Voltages
figure(1);
```

```
for i = 1:n_runs
    subplot(1,n_runs,i); % Choose top row
    hold on;
    for n = 1:length(out(i).vccout(1,:))
        plot(out(i).t, out(i).vccout(:,n));
    end
    hold off;
%     xlim([0 out(i).t(end)]);
%     ylim([0 ylims(1)]); % Set the same y-axis limits for each
    title('Module Voltages');
    legend('Vcc5', 'Vcc4', 'Vcc3', 'Vcc2' );
    ylabel('Vcc [V]');
    xlabel('Time [s]');
    grid on;
    grid minor;
end

% Plot the PnO duty cycle
figure(2);
for i = 1:n_runs
    subplot(1,n_runs,i); % Choose top row
    plot(out(i).t, out(i).D);
    ylabel('Duty Cycle');
    xlabel('Time [s]');
    grid on;
    grid minor;
end

% Plot the PV power for a D sweep
%{
figure(3);
ymax = 0;
for i = 1:n_runs
    temp = ceil(max(out(i).Ppv)/20)*20;
    if temp > ymax
        ymax = temp;
```

```
    end
end
for i = 1:n_runs
    subplot(1, n_runs, i);
    plot(out(i).t, out(i).Ppv);
    title(['P: initialD = ' num2str(testVals(i))]);
    xlabel('Time [s]');
    ylabel('Power [W]');
    grid minor;
    ylim([0 ymax]);
end
%}

% Plot power
figure(3);
ymax = 0;
for i = 1:n_runs
    temp = ceil(max(out(i).Ppv)/20)*20;
    if temp > ymax
        ymax = temp;
    end
end
for i = 1:n_runs
    subplot(1, n_runs, i);
    plot(out(i).t, out(i).Ppv);
    title('Power over Time');
    xlabel('Time [s]');
    ylabel('Power [W]');
    grid minor;
    ylim([0 ymax]);
end
```

Plot Irradiance

```
%{
figure(6)
```

```
plot(I9(:,1),I9(:,2),...
    I11(:,1),I11(:,2),...
    I13(:,1), I13(:,2), ...
    I15(:,1), I15(:,2), ...
    I17(:,1), I17(:,2), ...
    I19(:,1), I19(:,2));
legend('I9', 'I11', 'I13', 'I15', 'I17', 'I19');
grid minor;
title('Irradiance Data');
xlabel('Time [s]');
ylabel('Irradiance [W/m^2]');
%}
```

Shuffle converter algorithm

This code should be in the simulink model, but is included here for convenience

```
%{
function [D,err] = getD(Vin1, Vin2, nCells1, nCells2, Kp)
% Calculates the appropriate duty cycle value to equalise the cell voltages
% of two PV strings.
% Increase in D => decrease in Vin1
%               => increase in Vin2
% In a firmware implementation, values may need to be altered in
% consideration for timing, dead time, etc.

%Calculate error between scaled voltages
err = Vin1/nCells1 - Vin2/nCells2;

if Kp == -1 % Enter mode for static D
    D = 0.5;
    return
end

Vin = Vin1/nCells1 + Vin2/nCells2; % DC/DC converter input
Vout = Vin1/nCells1; % DC/DC converter output
```

```
D = Vout/Vin;% Calculate current D
% In the future, a direct feedback loop may be used, but this is the
% current solution

if (D > 0) && (D < 1) % If valid
    D = D + Kp*err;
end

if D > 1 % Set upper limit to 0
    D = 1;
elseif D < 0 % Set lower limit to 0
    D = 0;
end
end
%}
```

Remap data

This function returns data in a that is scaled by velocity v and scaled to time vector t. The starting point can be shifted by s seconds

```
function c = map2time(a, t, v, s)

[~, idx_unique] = unique(a(:,1));
b = [a(idx_unique,1), a(idx_unique,2)];

secs = (b(:,1)-s)*1000/v; % Convert mapping from Irr/km to Irr/
b_interp = interp1(secs, b(:,2), t, 'makima'); % can change method depending on shape
c = [t; b_interp]';

end
```